

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-06 (Unidad 3 - Tarea 2)

**Manipular bases de datos utilizando SGBD para la gestión de la información
almacenada (DML)**

MIEMBROS DEL EQUIPO:

- Líder: Jorge Juan Saldarriaga
- Miembro: José Julián Doria Ricardo
- Miembro: Nicolás Duque Serna

Informe con resultados

Equipo "4"

(TIA-6: Unidad 3 - Tarea 2)

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

1.- Inventario final de tablas del sistema de información

A continuación, se presenta el inventario definitivo de tablas de la base de datos apícola, corregido y mejorado respecto al modelo físico entregado en la TIA-04. Se incorporaron tablas faltantes para poder realizar operaciones DML completas, se corrigieron nombres siguiendo snake_case y se añadió la tabla consumidor, mercado implícito en ubicacion, y lote_produccion para soportar el detalle de pedidos correctamente.

Cuadro 1. Inventario de Tablas definitivas de la Base de Datos

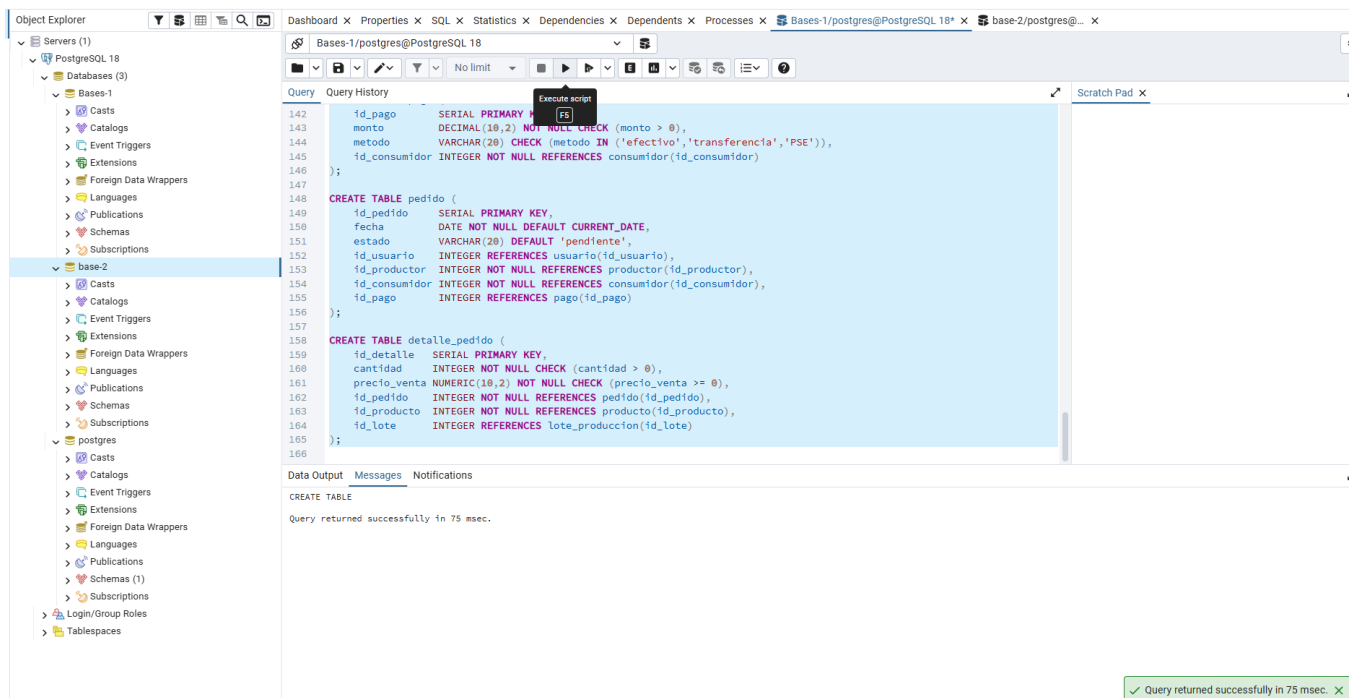
Nro	Tabla	Descripción	Tipo	Tablas Relacionadas
1	usuario	Almacena los usuarios del sistema	E	18
2	rol	Define los roles disponibles en el sistema	E	18
3	usuario_rol	Tabla intermedia que asocia usuarios con roles	R	1, 2
4	productor	Registro de productores apícolas del sistema	E	1
5	entidad_apoyo	Entidades financiadoras o de apoyo al productor	E	-
6	credito	Créditos otorgados a productores por entidades de apoyo	R	4, 5
7	ubicacion	Coordenadas geográficas y datos de los apiarios	E	-
8	apiario	Apiarios registrados y administrados por productores	E	4, 7

Nro	Tabla	Descripción	Tipo	Tablas Relacionadas
9	colmena	Colmenas ubicadas dentro de cada apiario	E	8
10	sensor	Sensores IoT instalados en las colmenas	E	9
11	registro_ambiental	Registros de datos ambientales captados por sensores	E	10
12	evento	Eventos registrados en el sistema apícola	E	1
13	sensor_evento	Tabla intermedia que asocia sensores con eventos	R	10, 12
14	inspeccion	Inspecciones realizadas a las colmenas	E	1, 9
15	certificacion	Certificaciones obtenidas tras inspecciones	E	14
16	producto	Catálogo de productos apícolas disponibles	E	-
17	productor_producto	Tabla intermedia que asocia productores con productos	R	4, 16
18	oferta_trabajo	Ofertas de trabajo publicadas en el sistema	E	16
19	lote_produccion	Lotes de producción generados por apiario-producto	E	8, 16
20	consumidor	Personas registradas que realizan pedidos	E	-
21	pago	Pagos asociados a los pedidos realizados	E	20
22	pedido	Pedidos realizados por usuarios en el sistema	E	1, 4, 20, 21
23	detalle_pedido	Detalle de los productos incluidos en cada pedido	R	16, 19, 22

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2.- Script de "creación" de las tablas definitivas de la Base de Datos (CREATE)

Se implementó el script de creación completo en PostgreSQL 18.1 utilizando pgAdmin 4. Las tablas se crearon respetando el orden de dependencias: primero las tablas independientes (sin claves foráneas) y luego las dependientes. Se utilizaron restricciones CHECK, UNIQUE, NOT NULL, y el tipo JSONB para la tabla apiario (campo ubicacion_geografica), lo cual permite almacenar datos semi-estructurados compatibles con escenarios de Big Data e IoT.



The screenshot displays the pgAdmin 4 interface with the PostgreSQL 18.1 query editor. The left sidebar shows the 'Object Explorer' with a tree view of the database structure, including 'Servers (1)', 'PostgreSQL 18', 'Databases (3)', 'Bases-1', and 'base-2'. The main pane shows a SQL script with the following content:

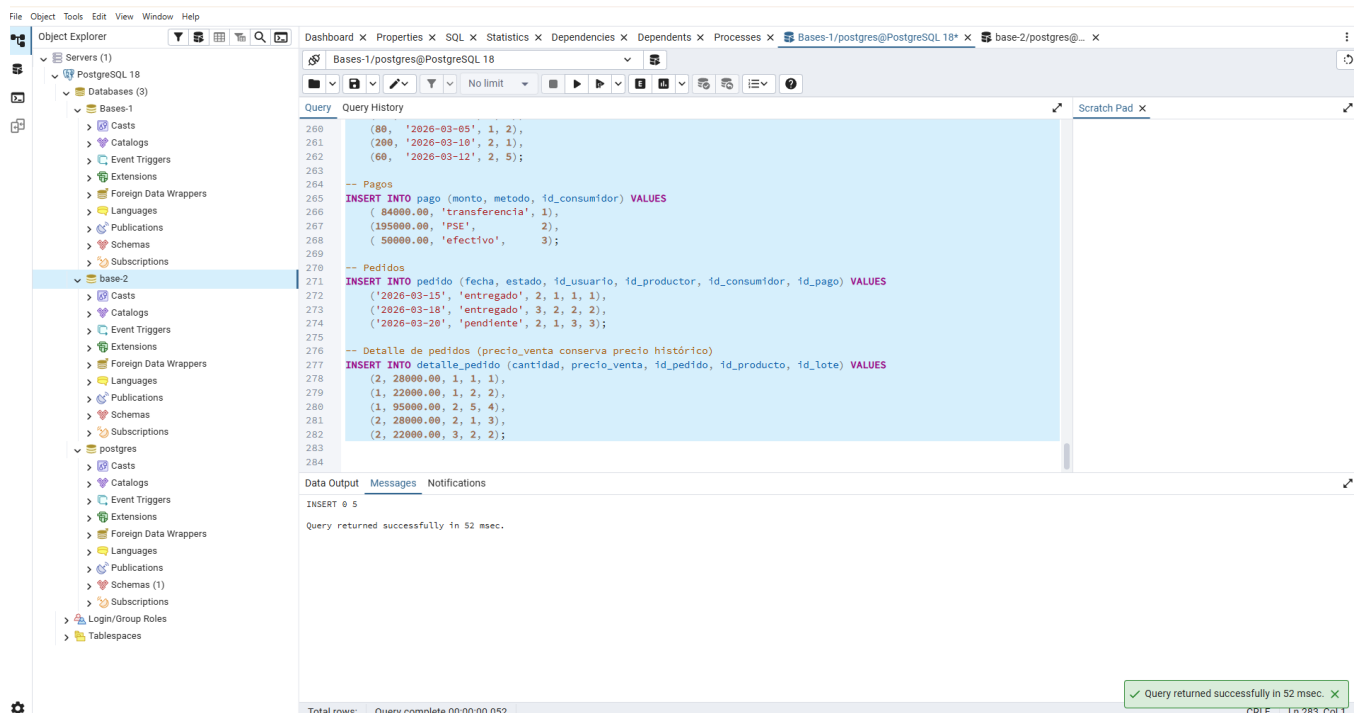
```
142     id_pago          SERIAL PRIMARY KEY,
143     monto            DECIMAL(10,2) NOT NULL CHECK (monto > 0),
144     metodo           VARCHAR(20) CHECK (metodo IN ('efectivo','transferencia','PSE')),
145     id_consumidor    INTEGER NOT NULL REFERENCES consumidor(id_consumidor)
146 );
147
148 CREATE TABLE pedido (
149     id_pedido        SERIAL PRIMARY KEY,
150     fecha            DATE NOT NULL DEFAULT CURRENT_DATE,
151     estado           VARCHAR(20) DEFAULT 'pendiente',
152     id_usuario       INTEGER REFERENCES usuario(id_usuario),
153     id_producto      INTEGER NOT NULL REFERENCES producto(id_producto),
154     id_consumidor    INTEGER NOT NULL REFERENCES consumidor(id_consumidor),
155     id_pago          INTEGER REFERENCES pago(id_pago)
156 );
157
158 CREATE TABLE detalle_pedido (
159     id_detalle       SERIAL PRIMARY KEY,
160     cantidad         INTEGER NOT NULL CHECK (cantidad > 0),
161     precio_venta     NUMERIC(10,2) NOT NULL CHECK (precio_venta >= 0),
162     id_pedido        INTEGER NOT NULL REFERENCES pedido(id_pedido),
163     id_producto      INTEGER NOT NULL REFERENCES producto(id_producto),
164     id_lote          INTEGER REFERENCES lote_produccion(id_lote)
165 );
166
```

The 'Data Output' pane at the bottom shows the message: 'CREATE TABLE' and 'Query returned successfully in 75 msec.' A green status bar at the bottom right confirms: '✓ Query returned successfully in 75 msec. ✕'.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3.- Script de "poblamiento" de la Base de Datos (INSERT)

Se realizó el poblamiento de las tablas principales del sistema apícola con datos ficticios pero coherentes con la realidad colombiana. Se incluyeron apicultores de distintos municipios del Anexo B, productos apícolas del Anexo C, y pedidos con su detalle según el Anexo D. Además, se incluyó el campo JSONB en la tabla apiario para almacenar la ubicación geográfica completa con metadatos adicionales.



```
260 (60, '2026-03-05', 1, 2),
261 (200, '2026-03-10', 2, 1),
262 (60, '2026-03-12', 2, 5);
263
264
265 -- Pagos
266 INSERT INTO pago (monto, metodo, id_consumidor) VALUES
267 ( 84000.00, 'transferencia', 1),
268 (195000.00, 'PSE', 2),
269 ( 50000.00, 'efectivo', 3);
270
271 -- Pedidos
272 INSERT INTO pedido (fecha, estado, id_usuario, id_producto, id_consumidor, id_pago) VALUES
273 ('2026-03-15', 'entregado', 2, 1, 1, 1),
274 ('2026-03-18', 'entregado', 3, 2, 2, 2),
275 ('2026-03-20', 'pendiente', 2, 1, 3, 3);
276
277 -- Detalle de pedidos (precio_venta conserva precio histórico)
278 INSERT INTO detalle_pedido (cantidad, precio_venta, id_pedido, id_producto, id_lote) VALUES
279 (2, 28000.00, 1, 1, 1),
280 (1, 22000.00, 1, 2, 2),
281 (1, 95000.00, 2, 5, 4),
282 (2, 28000.00, 2, 1, 3),
283 (2, 22000.00, 3, 2, 2);
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Data Output Messages Notifications

INSERT 0 5

Query returned successfully in 52 msec.

Total rows: Query complete 00:00:00.052

✓ Query returned successfully in 52 msec. ✕

4.- Script de actualización de la información (UPDATE)

Se realizaron operaciones de actualización sobre la tabla ubicacion (simulando cambios de dirección de mercados) y sobre la tabla producto para reflejar cambios en los precios de los productos apícolas.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, including 'base-2' and 'postgres' databases. The main window shows a SQL query script with the following content:

```
298 UPDATE ubicacion SET vereda = 'Chapinero Alto'
299 WHERE municipio = 'Bogotá';
300 -- Justificación: actualización del sector del mercado en Bogotá
301
302 UPDATE ubicacion SET vereda = 'Centro Histórico'
303 WHERE municipio = 'Tunja';
304 -- Justificación: reubicación del punto de distribución en Tunja
305
306 -- Verificación de las actualizaciones de mercados
307 SELECT departamento, municipio, vereda FROM ubicacion;
308
309 -----
310 -- Tres actualizaciones de precio en tabla producto
311
312 UPDATE producto SET precio = 30000.00
313 WHERE nombre = 'Miel de abejas pura';
314 -- Justificación: incremento por alta demanda de temporada
315
316 UPDATE producto SET precio = 100000.00
317 WHERE nombre = 'Jalea real fresca';
318 -- Justificación: escasez por reducción en producción
319
320 UPDATE producto SET precio = 19500.00
321 WHERE nombre = 'Cera de abejas natural';
322
```

The 'Data Output' tab shows the results of the queries:

id_producto [PK] integer	nombre character varying (30)	precio numeric (10,2)
1	Miel de abejas pura	30000.00
2	Polen apícola seco	22000.00
3	Propóleo en tintura	35000.00
4	Cera de abejas natural	19500.00
5	Jalea real fresca	100000.00
6	Miel con propóleo	32000.00

A status bar at the bottom indicates: 'Successfully run. Total query runtime: 77 msec. 6 rows affected.'

5.- Script de eliminación de la información (DELETE)

Se realizaron dos operaciones de eliminación: primero se insertó un producto por error y luego se eliminó; segundo, se insertó un productor que finalmente no participará en la red apícola y también se eliminó.

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure. The main window shows a SQL script with the following content:

```
330 -- Verificar que se eliminó (resultado esperado: 0 rows)
331 SELECT * FROM producto WHERE nombre = 'Vinagre de miel';
332
333 -- Caso 2: Productor ingresado por error
334
335 INSERT INTO usuario (nombre, email, id_rol)
336 VALUES ('Prueba Temporal', 'temp.productor@test.co', 2);
337
338 INSERT INTO productor (nombre, telefono, id_usuario)
339 VALUES ('Apícola Prueba - Temporal', '3000000000',
340 (SELECT id_usuario FROM usuario WHERE email = 'temp.productor@test.co'));
341
342 -- Verificar que se insertó
343 SELECT * FROM productor WHERE nombre LIKE '%Prueba%';
344
345 -- Eliminar el productor (ya no participará en la Red Apícola)
346 DELETE FROM productor WHERE nombre = 'Apícola Prueba - Temporal';
347 DELETE FROM usuario WHERE email = 'temp.productor@test.co';
348
349 -- Verificar eliminaciones (resultado esperado: 0 rows en ambos)
350 SELECT * FROM productor WHERE nombre LIKE '%Prueba%';
351 SELECT * FROM usuario WHERE email = 'temp.productor@test.co';
352
353
354
```

The 'Data Output' tab shows the results of the queries:

id_usuario [PK] integer	nombre character varying (30)	email character varying (50)	id_rol integer
-------------------------	-------------------------------	------------------------------	----------------

A status bar at the bottom indicates: 'Successfully run. Total query runtime: 107 msec. 0 rows affected.'

6.- Script de consultas de listados (SELECT)

Se implementaron cinco consultas SELECT con distinto nivel de complejidad, desde simples hasta con múltiples JOIN, siguiendo los requerimientos de la tarea.

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database structure, including servers, databases, and schemas. The main pane shows a SQL query with comments and syntax. The query is a complex SELECT statement with multiple JOINs and an ORDER BY clause. The results pane at the bottom shows the output of the query, which is a table with 8 columns and 2 rows.

```
-- Listar municipios con sus productores y apíarios
SELECT u.departamento,
       u.municipio,
       p.nombre AS productor,
       a.nombre AS apiario
FROM   ubicacion u
JOIN   apiario a ON a.id_ubicacion = u.id_ubicacion
JOIN   productor p ON p.id_productor = a.id_productor
ORDER BY u.departamento, u.municipio, p.nombre;

-----
-- Consulta #4: Con 3 JOIN
-- Listar productores, apiarios y productos que elaboran y venden
SELECT p.nombre AS productor,
       a.nombre AS apiario,
       u.municipio,
       pr.nombre AS producto,
       pr.precio
FROM   productor p
JOIN   apiario a ON a.id_productor = p.id_productor
JOIN   productor_producto pp ON pp.id_productor = p.id_productor
JOIN   producto pr ON pr.id_producto = pp.id_producto
LEFT JOIN ubicacion u ON u.id_ubicacion = a.id_ubicacion
ORDER BY p.nombre, a.nombre, pr.nombre;
```

id_pedido	fecha	estado	id_productor	productor	id_consumidor	consumidor	municipio
1	2026-03-...	pendiente	1	Apicola Montoya - Carlos Mont...	3	Sandra Millán	Medellín
2	2026-03-...	entregado	1	Apicola Montoya - Carlos Mont...	1	María Fernanda López	Medellín

Total rows: 2 Query complete 00:00:00.059

7.- Script de consultas con agrupaciones (GROUP BY y HAVING)

Se implementaron cinco consultas con agrupaciones utilizando GROUP BY, HAVING y funciones de agregación (COUNT, SUM, MAX, MIN, AVG), acompañadas de las respuestas a las preguntas analíticas requeridas.

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database structure. The main pane shows two SQL queries. The first query is a SELECT statement with a GROUP BY clause and a HAVING clause. The second query is a SELECT statement with a GROUP BY clause and an ORDER BY clause. The results pane at the bottom shows the output of the first query, which is a table with 2 columns and 1 row.

```
-- ¿Cuál fue el producto que recibió menos pedidos?
SELECT pr.nombre,
       SUM(dp.cantidad) AS total
FROM   detalle_pedido dp
JOIN   producto pr ON pr.id_producto = dp.id_producto
GROUP BY pr.nombre
ORDER BY total ASC
LIMIT 1;

-- ¿Cuál fue el municipio con el mayor monto (COP) de pedidos?
SELECT u.municipio,
       SUM(dp.cantidad * dp.precio_venta) AS monto_COP
FROM   detalle_pedido dp
JOIN   pedido pe ON pe.id_pedido = dp.id_pedido
JOIN   productor p ON p.id_productor = pe.id_productor
JOIN   apiario a ON a.id_productor = p.id_productor
JOIN   ubicacion u ON u.id_ubicacion = a.id_ubicacion
GROUP BY u.municipio
ORDER BY monto_COP DESC
LIMIT 1;
```

municipio	monto_cop
Bogotá	151000.00

Total rows: 1 Query complete 00:00:00.115

8.- Script de consultas con vistas (VIEW)

Se construyó una vista que consolida información relevante del sistema apícola: los apiarios con sus productores, municipios, número de pedidos recibidos y el monto total vendido en COP, filtrando solo aquellos que superen los 40.000 COP en ventas.

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database structure with 'base-2' selected. The main pane shows a SQL query that calculates summary statistics for apiarios and filters results based on a minimum sales amount of 40,000 COP. The query uses aggregate functions like COUNT, SUM, and AVG, and includes JOINs to combine data from different tables. The results pane shows a single row of data for an apiario in Antioquia.

```
662      COUNT(DISTINCT pe.id_pedido) AS total_pedidos,
663      SUM(dp.cantidad * dp.precio_venta) AS monto_total_COP,
664      AVG(dp.precio_venta) AS precio_promedio
665 FROM detalle_pedido dp
666 JOIN pedido pe ON pe.id_pedido = dp.id_pedido
667 JOIN productor p ON p.id_productor = pe.id_productor
668 JOIN apiario a ON a.id_productor = p.id_productor
669 JOIN ubicacion u ON u.id_ubicacion = a.id_ubicacion
670 GROUP BY u.departamento, u.municipio, a.nombre, p.nombre
671 HAVING SUM(dp.cantidad * dp.precio_venta) > 40000
672 ORDER BY monto_total_COP DESC;
673
674 -- =====
675 -- Consultar la vista completa
676 SELECT * FROM vista_ventas_apiario;
677
678 -- Consulta adicional sobre la vista: filtrar solo Antioquia
679 SELECT * FROM vista_ventas_apiario
680 WHERE departamento = 'Antioquia';
681
682
683
684
685
686
```

departamento	municipio	apiario	productor	total_pedidos	monto_total_COP	precio_promedio
Antioquia	Medellin	Apiario La Montaña	Apícola Montoya - Carlos Mont...	2	122000.00	24000.000000000000

Successfully run. Total query runtime: 96 msec. 1 rows affected.

9.- Script de consultas con parámetros (PREPARE)

Se implementó una consulta parametrizada con tres parámetros utilizando PREPARE y EXECUTE en PostgreSQL. La consulta filtra pedidos por departamento, municipio y un monto mínimo de ventas, permitiendo reutilizar la consulta de forma eficiente.

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database structure with 'base-2' selected. The main pane shows a SQL script that prepares a query with three parameters (department, municipality, and minimum sales amount) and then executes it twice with different values. The results pane shows the output of the EXECUTE statement, indicating successful execution.

```
695 JOIN pedido pe ON pe.id_pedido = dp.id_pedido
696 JOIN consumidor c ON c.id_consumidor = pe.id_consumidor
697 JOIN productor p ON p.id_productor = pe.id_productor
698 JOIN apiario a ON a.id_productor = p.id_productor
699 JOIN ubicacion u ON u.id_ubicacion = a.id_ubicacion
700 WHERE u.departamento = $1
701 AND u.municipio = $2
702 GROUP BY u.departamento, u.municipio, p.nombre, c.nombre
703 HAVING SUM(dp.cantidad * dp.precio_venta) >= $3
704 ORDER BY monto_COP DESC;
705
706 -- =====
707 -- Ejecutar con parámetros: Antioquia / Medellín / mínimo $50.000
708 EXECUTE consulta_pedidos_filtrada('Antioquia', 'Medellin', 50000);
709
710 -- Ejecutar con parámetros: Cundinamarca / Bogotá / mínimo $30.000
711 EXECUTE consulta_pedidos_filtrada('Cundinamarca', 'Bogotá', 30000);
712
713 -- Liberar la consulta preparada cuando ya no se necesite
714 DEALLOCATE consulta_pedidos_filtrada;
715
716
717
718
719
```

DEALLOCATE

Query returned successfully in 51 msec.

10.- Script de verificación de propiedades ACID

Se verificaron las cuatro propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) mediante transacciones con BEGIN, COMMIT y ROLLBACK en PostgreSQL, demostrando la robustez y confiabilidad del modelo implementado.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

11.- Análisis de resultados

La implementación del modelo físico mejorado en PostgreSQL 18.1 permitió ejecutar de forma exitosa todas las operaciones DML requeridas. Uno de los principales aprendizajes fue entender que un buen modelo conceptual y lógico no garantiza por sí solo un modelo físico funcional: fue necesario revisar las relaciones entre tablas, corregir claves foráneas que estaban incompletas, añadir la tabla consumidor (que estaba ausente en el TIA-04) y crear la tabla lote_produccion para poder conservar los precios históricos en el detalle de pedidos.

El uso del tipo JSONB en el campo ubicacion_geografica de la tabla apiario fue una decisión técnica relevante, ya que permite almacenar datos jerárquicos semiestructurados (coordenadas, departamento, municipio, vereda, flora dominante) sin necesidad de normalizar en múltiples tablas. Esto es especialmente útil en escenarios de IoT y Big Data, donde los sensores pueden generar datos con formatos variables. PostgreSQL permite consultar este campo con operadores propios (->> y ->), lo que facilita filtros eficientes sobre campos anidados.

La verificación de propiedades ACID demostró la solidez del modelo. La Atomicidad garantizó que una transacción con ROLLBACK no dejara cambios parciales. La Consistencia fue evidente cuando el motor rechazó inserciones con claves foráneas inválidas y precios negativos. El Aislamiento permitió comprobar que los cambios no confirmados no son visibles a otras sesiones. La Durabilidad confirmó que los registros con COMMIT persisten de forma permanente.

En cuanto a las consultas, las operaciones con múltiples JOIN mostraron la importancia de diseñar bien las relaciones desde el inicio. Las vistas y las consultas preparadas ofrecen ventajas importantes para aplicaciones reales: las vistas simplifican consultas complejas y las consultas con PREPARE mejoran el rendimiento al reutilizar planes de ejecución optimizados.

12.- Reflexiones y conclusiones individuales

Conclusión individual – Nicolás Duque Serna

Esta tarea me permitió comprender de manera práctica la diferencia entre diseñar una base de datos y hacerla realmente funcional. En el TIA-04 pensamos que el modelo estaba completo, pero al intentar insertar datos reales nos dimos cuenta de que faltaba la tabla consumidor y que varias relaciones entre tablas no estaban bien definidas. Corregir esos errores antes de empezar con el DML fue un paso obligatorio y muy valioso. Lo que más me llamó la atención fue el tipo de dato JSONB: nunca lo había usado y me parece muy útil para situaciones donde los datos no tienen siempre la misma estructura, como pasa con los sensores IoT en los apiarios. También entendí por qué el precio de venta se debe guardar en el detalle del pedido y no tomarlo directamente del catálogo de productos: el precio puede cambiar con el tiempo y siempre hay que conservar el valor original de la transacción. En mi formación como desarrollador, este tipo de experiencias prácticas vale mucho más que solo leer teoría.

Conclusión individual – Jorge Juan Saldarriaga

Lo que más me aportó esta tarea fue ver en tiempo real cómo las propiedades ACID protegen la integridad de los datos. Hacer el ROLLBACK y verificar en dos ventanas simultáneas que los valores volvían a su estado original fue una demostración muy clara de algo que antes solo entendía de forma abstracta. También reforcé mi comprensión de los JOIN: saber en qué orden conectar las tablas, qué tipo de JOIN usar (INNER, LEFT) y cómo afecta eso a los resultados de la consulta es fundamental para cualquier desarrollador de software. El trabajo en equipo también fue importante: coordinar quién hacía cada sección del informe y quién ejecutaba cada script evitó errores y nos permitió avanzar más rápido. A nivel profesional, sé que estas habilidades son las que se usan en el día a día en una empresa real.

Conclusión individual – José Julián Doria Ricardo

Este trabajo me hizo entender que la manipulación de datos es mucho más que escribir `SELECT * FROM tabla`. Aprendí que una consulta mal escrita puede devolver datos incorrectos, aunque no genere errores de sintaxis, y que el orden de los JOIN y los filtros WHERE son decisivos para obtener información de calidad. La parte de las consultas con GROUP BY y HAVING me pareció especialmente útil porque permite responder preguntas de negocio reales: ¿cuál productor recibió más pedidos?, ¿cuál municipio vendió más?. Estas son preguntas que cualquier gerente o administrador querría responder. También me gustó la parte de las vistas, porque permiten encapsular consultas complejas y darles un nombre fácil de recordar. En general, siento que este trabajo me dejó una base sólida para seguir aprendiendo sobre bases de datos y aplicarlo en proyectos reales.